

Product Requirements Document

Truv ↔ Blue Sage LOS + LION POS Integration

Pay-Per-Funded-Loan Billing Architecture

Author: Andrew Toth

Role: Lead PM, Implementation — Mortgage BU (candidate)

Date: June 2026

Status: Draft v1.0 — for engineering review

Classification: Confidential / Hiring Use Only

0. Document Overview

Purpose

This PRD defines the complete integration contract for two parallel workstreams: deepening the existing Blue Sage LOS integration from one-click order initiation into a full bidirectional flow including a net-new loan-funded notification mechanism; and standing up a new LION integration using Truv Embedded Orders.

Primary use case: Income verification (VOIE)

The primary verification use case covered by this integration is income and employment verification (VOIE) via Truv Bridge. This is the core product used in wholesale and correspondent mortgage origination for GSE-eligible loans — the verified income data (voie_report_id) auto-populates DU® and LPA® AUS fields in Blue Sage and enables GSE-approved automated underwriting.

This integration is scoped to income verification (products:["income"]). The Truv platform supports additional products (employment, assets, transactions) and document upload fallback within the same order session, but these are out of scope for this initial implementation and can be added via template configuration in a future phase.

PCL billing and multi-product orders: Under the pay-per-funded-loan commercial model, Truv invoices the lender exactly once per funded loan, regardless of how many verification products ran on the underlying order. A loan with VOIE + VOA + a refresh cycle still generates a single PCL billing event when the loan funds. The funded notification schema does not require a products[] field — billing is loan-level, not product-level. This is enforced by the composite billing index {client_id}:{loan_number} in Section 4 — client_id prevents cross-lender collision in multi-lender BSS deployments and is a v1.0 requirement.

Scope boundary

In scope: Integration architecture, API contract design, order orchestration, webhook handling, report retrieval, the loan-event contract between Blue Sage and Truv, and this PRD that engineering teams on both sides build against.

Out of scope: The borrower experience inside Truv Bridge — branding, copy, conversion optimization, and the verification UI itself — is owned by a separate PM. Bridge is treated as a black box called via bridge_token.

Document structure

Sections 1–9 map directly to the PRD requirements in the case study brief:

- 1. Integration Architecture — system participants, sequence diagrams, end-to-end flow paths
- 2. API Surface by Team — every endpoint, method, parameters, response shapes, error codes, rate limits — grouped by LION POS vs Blue Sage LOS
- 3. Webhook Contracts — both directions, signature verification, retry policy, idempotency
- 4. Loan-Funded Notification Design — the net-new PCL billing trigger; webhook vs REST rationale; full schema
- 5. Loan-State Taxonomy and Billing Mapping — Blue Sage loan states mapped to PCL billing actions
- 6. Error Handling and Reconciliation — all failure scenarios with backstop mechanisms
- 7. Sandbox Test Plan — nine test cases mapped to specific Truv sandbox credentials

- 8. Go-Live Checklist — pre-prod validation, owners, go/no-go criteria, rollback plan
- 9. Open Questions — genuine unknowns requiring Truv/Blue Sage engineering confirmation

1. Integration Architecture

1.1 System participants

- LION POS — borrower-facing point of sale; hosts the Truv Bridge embed; creates and monitors Truv orders.
- Truv Platform — verification engine; manages orders, Bridge sessions, webhooks, report generation, and (after this PRD) billing event tracking.
- Blue Sage LOS — loan processing and closing system; receives VOIE data; submits to AUS (DU/LPA); sends loan-funded notifications to Truv.
- Borrower — authenticates via Truv Bridge embedded in LION; connects payroll account(s).
- Loan Officer / Processor — works in Blue Sage; monitors verification status; initiates refresh if needed; closes loan.

1.2 Integration sequence — all paths

Four integration paths with continuous step numbering. Actor column color key: LION (blue), Truv Platform (green), Blue Sage LOS (red), Borrower (navy).

Step	Actor	Message / Action	Notes
Path A — Borrower verification (LION → Truv → Blue Sage)			
1	LION → Truv	POST /v1/orders/	Body: products:["income"], loan_number, first/last name. Returns order_id + bridge_token.
2	Truv → LION	HTTP 201 { order_id, bridge_token, user_id, share_url }	bridge_token is short-lived; pass to TruvBridge.init() client-side only. Never log or store server-side.
3	LION → Borrower	TruvBridge.init({ isOrder:true, bridgeToken })	Bridge launches in LION iframe. All borrower UX, employer search, MFA, and doc fallback are handled by Truv — BLACK BOX.
4	Borrower → Truv	Authenticates with payroll provider(s)	Multi-employer connections supported

			within one Bridge session. Doc upload fallback for gig/self-employed borrowers.
5	Truv → LION	Webhook: task-status-updated (per employer)	Payload: task_id, link_id, status [done error mfa_required], employer_name. POS state signal only — not a billing event.
6	Truv → LION	Webhook: order-status-updated (status=completed)	Payload: order_id, user_id, link_id, status, data_source. Triggers LION to retrieve full order data.
7	LION → Truv	GET /v1/orders/{order_id}/	LION retrieves: voie_report_id, employers[], is_suspicious. Used for POS display only — not relayed to Blue Sage.
8	Truv → Blue Sage	Direct report delivery (existing integration)	Truv delivers voie_report_id, pdf_report, pay stubs, W-2s directly to Blue Sage Findings tab and Document Manager. LION is NOT a relay.
—	Blue Sage	LO reviews Findings tab; submits to AUS	voie_report_id auto-populates DU® / LPA® fields. Existing Blue Sage behavior — no new Blue Sage build required for report delivery.
Path B — Loan state and PCL billing ★ Net-new			
9	Blue Sage	Loan progresses through states	originated → processing → underwriting → clear_to_close. No Truv events at these stages. BSS literal status field names TBC — see Section 10 Note 2.
10	Blue Sage	★ Loan funds — note executed	Distinct status transition in Blue Sage; MIN

			transfer event; discrete auditable funded_at timestamp.
11	Blue Sage → Truv	Webhook: loan.funded (order_id, loan_number, funded_at, idempotency_key)	★ Primary design: Blue Sage event framework publishes to Truv webhook subscription. Fallback: REST POST /v1/orders/{id}/funded/ — see Section 4.1.2.
12	Truv	Validate auth → look up order_id → check idempotency store	401 = invalid X-BSS-Webhook-Sign or invalid credentials 404 = order_id not found 409 = Gate A or Gate B duplicate
13	Truv → Billing Store	CREATE billing event { client_id+loan_number (composite PK), order_ids:["ord_latest",...,"ord_original"] (most-recent first), funded_at, status:pending }	Store idempotency_key (TTL 90 days); index {client_id}:{loan_number} → billing_event_id; order_ids[] populated with triggering order_id.
14	Truv → Blue Sage	HTTP 200 – billing event created	
15	Blue Sage → Truv	Webhook: loan.reversed (funded:false, reversal_reason)	Piggyback closed-end second fires with a different loan_number → distinct {client_id}:{loan_number} composite key → 2nd billing event (2x PCL confirmed assumption).
16	Truv	VOID billing event → status:voided	
Path C — Refresh verification (same order_id — no new PCL event)			
17	Blue Sage	LO triggers refresh pre-closing	e.g. employment gap or data staleness detected before closing.
18	Blue Sage → Truv	POST /v1/orders/{id}/refresh/ (products:["employment"], employer_id)	Same order_id retained. No new PCL billing event triggered.

19	Truv → Borrower	New Bridge session sent to borrower	Borrower re-authenticates via new bridge_token.
20	Truv → LION	Webhook: order-status-updated (completed)	LION updates POS state. Truv delivers refreshed report to Blue Sage directly.
Path D — Reconciliation backstop (if funded notification missed)			
21	Truv	Poll T+24h: order completed with no funded event received	Safety net for missed or failed Blue Sage webhook delivery.
22	Truv → Blue Sage	GET /bluesage/loans/{loan_number}/status	★ Confirm with Blue Sage eng: does this endpoint exist or need to be built?
23	Blue Sage → Truv	{ loan_number, status, funded_at, order_id }	Truv creates billing event if status=funded.

1.3 End-to-end flow — paths reference

Path A: Borrower verification (LION → Truv → LION → Blue Sage)

1. LION backend calls POST /v1/orders/ with borrower PII, products=["income"], loan_number, and optional employer prefill. Truv returns order_id + bridge_token.
2. LION frontend initializes TruvBridge.init({ isOrder: true, bridgeToken }) and presents the embedded Bridge to the borrower.
3. Borrower authenticates with payroll provider(s) inside Bridge. Truv handles MFA, employer search, and multi-employer connections within the single order session.
4. Truv fires task-status-updated (per employer) and order-status-updated (status=completed) webhooks to LION's registered endpoint.
5. LION backend calls GET /v1/orders/{order_id}/ to retrieve the full verification data package including voie_report_id. LION uses this to update its own loan application state — marking verification complete in the POS and releasing the file for processing.
6. Truv delivers the VOIE report, supporting documents (pay stubs, W-2s), and voie_report_id directly into Blue Sage via the existing Truv↔Blue Sage integration — populating the Findings tab and Document Manager. LION does not relay data to Blue Sage; Truv handles both delivery paths independently.
7. Blue Sage loan officer reviews the verified data in the Findings tab and triggers AUS submission — voie_report_id auto-populates DU® or LPA® fields. This is existing Blue Sage behavior; no new build required on the Blue Sage side for report delivery.

Path B: Loan-state events (Blue Sage → Truv)

8. Blue Sage loan progresses through origination → underwriting → clear-to-close. No Truv notifications at these stages.
9. Loan funds. Blue Sage triggers POST /v1/orders/{order_id}/funded/ with the funded notification payload (see Section 4). This is the net-new mechanism.
10. Truv receives the funded notification, validates auth, looks up order_id (metadata), checks Gate A (idempotency_key — exact-retry guard) then Gate B ({client_id}:{loan_number} composite — structural dedup, client_id from auth header), creates a PCL billing event, and returns HTTP 200.
11. If the loan is subsequently reversed, Blue Sage sends a second POST with funded=false and a reversal_reason. Truv voids the billing event.

Path C: Refresh verification (mid-process)

12. Loan officer in Blue Sage determines a VOIE refresh is needed (e.g., employment gap, data staleness before closing).
13. Blue Sage calls POST /v1/orders/{order_id}/refresh/ specifying products:["employment"] and the employer_id to refresh.
14. Truv triggers a new Bridge session for the borrower. order-status-updated fires on completion.
15. Refresh does not create an additional billing event — PCL remains tied to the funded notification only.

Path D: Reconciliation backstop (if funded notification missed)

16. If Truv receives no funded notification within 24 hours of an order reaching completed status, Truv's reconciliation backstop activates.
17. Truv polls a Blue Sage loan status endpoint: GET /bluesage/loans/{loan_number}/status.
18. If the returned status indicates the loan funded, Truv creates the billing event using the funded_at timestamp from the response.
19. This path is a safety net — primary delivery is via the loan.funded webhook (Path B). Blue Sage engineering must confirm whether this polling endpoint exists today or needs to be built.

1.4 System Ownership Matrix

Section 1.4 — System Ownership Matrix				
	OWNS / initiates	CONSUMES / receives	TRIGGERS	Not involved
Integration touchpoint	LION POS	Truv Platform	Blue Sage LOS	Borrower
POST /v1/orders/ (create order)	OWNS calls API	RECEIVES returns order_id + bridge_token	—	—
TruvBridge.init() (borrower UX)	OWNS injects iframe	OWNS Black box UX, MFA, doc fallback	—	INTERACTS authenticates payroll
task-status-updated (webhook)	CONSUMES POS state display	OWNS sends per-employer	—	—
order-status-updated (webhook)	CONSUMES triggers GET order	OWNS sends on complete	(Truv pushes directly to BSS)	—
GET /v1/orders/{id}/ (retrieve order)	OWNS POS state only	RECEIVES	CONSUMES reconciliation / refresh only	—
Report delivery (voie_report_id, PDFs)	— NOT a relay	OWNS pushes directly to BSS	CONSUMES Findings tab, Doc Manager, AUS	—
POST /v1/orders/{id}/refresh/ (refresh order)	—	RECEIVES	OWNS triggers refresh	CONSUMES re-authenticates
loan.funded webhook (BSS → Truv)	—	CONSUMES validates + creates billing event	OWNS triggers on loan status = funded	—
{client_id}:{loan_number} billing index	—	OWNS composite PK, dedup + store	TRIGGERS funded event is the source	—
Webhook registration POST /v1/webhooks/	OWNS registers endpoint at provisioning	RECEIVES stores URL + secret token	OWNS registers funded event subscription	—
Auth (API keys) X-Access-Client-Id	OWNS stores in secrets manager	VALIDATES on every call	OWNS stores in secrets manager	—

LION is NOT a relay — verification data flows Truv → LION (POS state) and Truv → BSS (report delivery) on independent channels. BSS → Truv flows only for funded events and refresh triggers.

2. API Surface by Team

2.1 Endpoint inventory

All endpoints use base URL <https://prod.truv.com/v1/> in production and <https://sandbox.truv.com/v1/> in sandbox. Authentication: X-Access-Client-Id and X-Access-Secret request headers on all server-to-server calls.

Team	Endpoint	Method	Purpose
LION	POST /v1/orders/	POST	Create VOIE order (products:["income"] for this integration). source=blue_sage. Returns order_id + bridge_token.
LION	GET /v1/orders/{id}/	GET	Retrieve completed order data + voie_report_id
LION	POST /v1/orders/{id}/refresh/	POST	Trigger re-verification on existing order
LION	POST /v1/orders/{id}/cancel/	POST	Cancel pending order
Blue Sage	GET /v1/orders/{id}/	GET	Retrieve order data for reconciliation or refresh management. Note: Truv delivers voie_report_id and documents to Blue Sage directly — Blue Sage does not act on order-status-updated to pull data.
Blue Sage	POST /v1/orders/{id}/funded/[NEW]	POST	Truv webhook subscription endpoint — receives loan.funded event from Blue Sage event framework (webhook design) OR fallback REST POST if Blue Sage webhook framework unavailable
Blue Sage	POST /v1/orders/{id}/funded/[NEW]	POST	Truv webhook or REST fallback — reversal event. Webhook: loan.status.reversed payload. REST: body { funded: false, reversal_reason }
Both teams	GET /v1/templates/	GET	Retrieve lender customization template list — Truv CS action at onboarding only
Both teams	GET /v1/company-mapping/	GET	Resolve employer name to Truv company_id before order creation

2.2 Key response shapes

GET /v1/orders/{id}/ — order-level fields

Consuming team legend: LION (blue) = POS display/state management only. Blue Sage (red) = AUS submission, Document Manager, underwriting. Both (green) = consumed by both teams.

Field	Type	Consuming team	Description / values
-------	------	----------------	----------------------

order_id	string	Both	Truv order identifier — primary key across all paths
status	string (enum)	LION	pending sent completed error canceled expired no_data skipped
data_source	string (enum)	LION	payroll docs financial_accounts — how borrower verified
voie_report_id	string	Blue Sage	★ GSE report ID — auto-populates DU® / LPA® AUS fields. Primary Blue Sage consume.
pdf_report	URI	Blue Sage	Employer-level PDF report URL → Document Manager
is_suspicious	boolean	Both	★ Fraud flag — GATE CONDITION. Per Truv docs: indicates potential document manipulation or irregularities requiring review — not necessarily definitive fraud. High-confidence fraud fails the task entirely (is_suspicious is not set). Triggers: browser-generated PDFs, altered documents, SSN mismatch. LION action: display LO warning, block auto-AUS submission, require manual escalation to underwriting. Blue Sage action: block AUS submission if true.
provider	string	LION	Payroll provider connected (adp, workday, paychex, etc.)
refresh_status	string	LION	Status of most recent refresh task — for POS display
employers[]	array	Both	One object per employer — see employer fields table below

GET /v1/orders/{id}/ — employers[] object fields

One object per employer connected. Core underwriting data lives here. Blue Sage consumes most fields for AUS; LION consumes display fields only.

Field	Type	Consuming team	Description / values
id	string	Blue Sage	Unique employer ID — use as employer_id in refresh calls POST /v1/orders/{id}/refresh/
job_title	string	LION	Employee job title from payroll system — POS display
job_type	string (enum)	Blue Sage	F=Full time P=Part time S=Seasonal C=Contract D=Daily — AUS field
start_date	date	Blue Sage	Employment start date — underwriting field

end_date	date null	Blue Sage	null = currently employed. Key UW data point.
is_active	boolean	Both	★ Active employment flag — LION displays; Blue Sage uses for AUS eligibility
income	decimal string	Blue Sage	Annual income figure from payroll — primary AUS income field
income_unit	string	Blue Sage	YEARLY MONTHLY WEEKLY HOURLY
pay_frequency	string (enum)	Blue Sage	M=Monthly B=Biweekly W=Weekly D=Daily S=Semi-monthly
company.name	string	LION	Employer name — POS display and order prefill
company.address	object	Blue Sage	Street, city, state, zip — AUS employer address field
profile.ssn	string	Blue Sage	Borrower SSN from payroll — Truv validates vs. SSN at order creation. Blue Sage audit.
statements[]	array	Blue Sage	Pay stubs: pay_date, gross_pay, net_pay, gross_pay_ytd, file (PDF URL) → Document Manager
w2s[]	array	Blue Sage	W-2 records: year, file (PDF URL) → Document Manager. Up to 3 years.
annual_income_summary[]	array	Blue Sage	Year-level rollover: year, gross_pay, regular, bonus, commission, overtime — AUS income history

LION consumption summary: order_id (→ GET /v1/orders/{id}/ for POS state), status, is_suspicious, employer.is_active, company.name, job_title. Blue Sage consumption summary: voie_report_id (AUS — VOIE), pdf_report + statements[].file + w2s[].file (Document Manager), income + job_type + annual_income_summary (underwriting). Truv delivers to Blue Sage directly via existing integration — LION does not relay. Both teams gate on is_suspicious=false before proceeding.

Webhook: order-status-updated (Truv → LION)

Field	Type	Consuming team	Description
event_type	string	LION	order-status-updated
order_id	string	LION	Truv order identifier
status	string (enum)	LION	pending sent completed error canceled expired no_data skipped
data_source	string	LION	payroll docs financial_accounts
user_id	string	LION	Truv user identifier

link_id	string	LION	Individual connection ID — use to retrieve link-level report if needed
---------	--------	------	--

Webhook: task-status-updated (Truv → LION only)

Field	Type	Consuming team	Description
event_type	string	LION	task-status-updated
task_id	string	LION	Individual task identifier
link_id	string	LION	Connection identifier
status	string (enum)	LION	done error mfa_required — LION uses for per-employer POS status display only
employer_name	string	LION	Employer name for this connection attempt

2.3 Auth model

- Server-to-server calls (order create/retrieve, funded notification): X-Access-Client-Id + X-Access-Secret headers. Keys scoped per lender; stored in secrets manager, never in environment variables. TLS 1.2+ required on all calls.
- Inbound webhooks from Truv → LION: signature in X-Webhook-Sign header (HMAC-SHA256 of request body + shared secret). LION must verify before processing. The shared secret is returned when LION registers its webhook endpoint via POST /v1/webhooks/ (fields: name, webhook_url, events[], env_type). Also configurable in Truv Dashboard > Development > Webhooks. BSS→Truv funded notification uses three-layer auth: (1) X-BSS-Webhook-Sign HMAC-SHA256 payload signing — Truv verifies as Step 1; (2) X-Access-Client-Id + X-Access-Secret credential auth; (3) TLS 1.2+. Signing secret provisioned by Truv CS at onboarding. See Section 3.5.1.
- Bridge token (bridge_token): short-lived token scoped to one order session; passed to TruvBridge.init() client-side only. Never logged or stored server-side.

2.4 Rate limits and error codes

- Truv standard rate limit: 100 requests/minute per client_id. Order creation and funded notifications are low-frequency; no throttling concern in normal operation.
- HTTP 200 — success.
- HTTP 400 — validation error (missing required field, invalid funded_at format). Blue Sage must log and alert; do not retry without correcting payload.
- HTTP 404 — order_id not found. Blue Sage must escalate to Truv support; indicates a sync issue.
- HTTP 409 — idempotency conflict (funded event already recorded for this key). Return 409, not 200, so Blue Sage can distinguish duplicate from new. Log both sides.
- HTTP 429 — rate limit hit. Retry after Retry-After header value.
- HTTP 5xx — Truv internal error. Retry with exponential backoff (1m, 5m, 30m).

3. Webhook Contracts

3.1 Webhook inventory

Event	Direction	Trigger / Payload keys
order-status-updated	Truv → LION	status=completed order_id, user_id, link_id — retrieve order data
task-status-updated	Truv → LION	Per-employer connection status (Done / Error) link_id, task_id

Section 3 — Webhook Contract Cards	
Truv → LION POS Truv Platform → LION POS registered endpoint Events - order-status-updated - task-status-updated Auth & signing Method: HMAC-SHA256 X-Webhook-Sign header Secret: Returned by POST /v1/webhooks/ at registration Verify: HMAC-SHA256(raw_body, secret) == X-Webhook-Sign Retry policy Attempts: 3 (1m → 5m → 30m exponential backoff) On failure: Truv ops alerted after 3rd attempt Timeout: LION endpoint must respond < 1 second Idempotency Header: X-Webhook-Event-Id on every payload LION action: Store event IDs, discard duplicates (TTL 48h) Registration POST /v1/webhooks/ { webhook_url, events:[], env_type } Alt: Truv Dashboard → Development → Webhooks TLS: HTTPS + TLS 1.2+ on LION receiver endpoint	Blue Sage LOS → Truv ★ Net-new Blue Sage LOS → POST /v1/orders/{id}/funded/ Events - loan-funded - loan-reversed Same endpoint for both webhook trigger and REST fallback Auth — three layers (all required v1.0) 1. X-BSS-Webhook-Sign: HMAC-SHA256(payload, signing_secret) Payload integrity — Truv verifies FIRST (Step 1 in §4.4) Signing secret issued by Truv CS at onboarding 2. Credentials: X-Access-Client-Id + X-Access-Secret 3. Transport: TLS 1.2+ (all three layers = enterprise reliability) Retry policy (BSS outbound) On 5xx: 3 retries (1m → 5m → 30m) On 409: Do NOT retry — Gate A or Gate B duplicate On 4xx: Do NOT retry — fix payload or credentials first Idempotency Gate B PK: {client_id}:{loan_number} (header-derived — NOT in payload) Gate A key: {client_id}-{loan_number}-{date}-v1 (BSS-supplied, auditability) Trigger Event: BSS loan status → funded (wire disbursement) Mechanism: Event bus preferred; direct callout acceptable Frequency: Once per {client_id}:{loan_number} (piggyback = 2x)

3.2 Signature verification — inbound webhooks from Truv → LION only

- Truv signs all outbound webhooks (to LION) with HMAC-SHA256 using the lender's shared secret. This secret is returned when LION registers its endpoint via POST /v1/webhooks/ or Truv Dashboard. Note: BSS→Truv funded notification does NOT use this pattern — it is authenticated via X-Access-Client-Id + X-Access-Secret headers (server-to-server credential auth).
- Signature is in the X-Webhook-Sign request header.
- Receiving endpoint computes: HMAC-SHA256(raw_request_body, shared_secret) and compares to X-Webhook-Sign. Reject with 401 if mismatch.
- Endpoint must respond with HTTP 200 within 1 second. Any non-200 response or timeout triggers Truv's retry logic.

3.3 Retry policy (Truv outbound)

- 3 retries with exponential backoff: 1 minute, 5 minutes, 30 minutes.
- After 3 failures, Truv ops is alerted. No further automatic retries.
- Each retry carries the same payload and the same webhook event ID — receiving endpoints must deduplicate on event ID.

3.4 Idempotency for inbound webhooks

- Both LION and Blue Sage must store processed webhook event IDs and discard duplicates. TTL: 48 hours.
- Webhook event ID is in the X-Webhook-Event-Id header on all Truv-originated webhooks.

3.5 Blue Sage → Truv: loan-funded notification webhook

This is the billing-critical webhook direction — Blue Sage publishes a funded event to Truv, triggering PCL billing. It is the only webhook that flows from Blue Sage to Truv. Full design rationale and fallback REST POST option are in Section 4.1. All four funded scenarios (happy path, duplicate, reversal, piggyback) are sequenced in Section 4.0.

3.5.1 Event details

Direction: Blue Sage LOS → Truv Platform

- Primary delivery: Blue Sage event framework publishes loan.funded to Truv's registered webhook subscription endpoint.
- Fallback delivery: if Blue Sage does not support outbound webhook subscriptions, Blue Sage makes a direct REST POST to Truv endpoint POST /v1/orders/{order_id}/funded/ — see Section 4.1.2.
- Trigger: Blue Sage loan status transitions to funded (note executed, disbursement wire sent).
- Frequency: once per funded loan per note. Piggyback structures (first lien + closed-end second) each fire a separate event with distinct loan_number values.
- Authentication — two layers, both required for v1.0: 1. Payload signing: Truv issues BSS a webhook signing secret at onboarding. BSS signs each loan.funded payload with HMAC-SHA256 and includes the signature in X-BSS-Webhook-Sign header. Truv verifies this before any other processing — rejects with 401 if mismatch. This provides payload integrity and is the foundation of enterprise-grade webhook reliability (equivalent to ICE/Encompass outbound webhook patterns). 2. Credential auth: X-Access-Client-Id + X-Access-Secret headers over TLS 1.2+. Blue Sage stores both credentials and signing secret in secrets manager — never in environment variables. Setup: Truv CS provisions BSS with client_id, access_secret, and webhook signing secret at onboarding. No endpoint registration required — BSS posts to the documented Truv endpoint.

3.5.2 Payload schema

Section 3.5.2 — loan.funded Payload Anatomy (BSS → Truv)

POST loan.funded webhook payload

```
{
  "event": "loan.funded"
  "order_id": "ord_abc123xyz"
  "loan_number": "2026-WC-008812"
  "funded": true
  "funded_at": "2026-06-03T14:22:00Z"
  "idempotency_key": "bss_wc_001-2026-WC-008812-2026-06-03-v1"
  "loan_type": "first_mortgage"
  "reversal_reason": null (omit if funded=true)
}
```

Field annotations

metadata

Most recent active order_id on the loan. Truv uses to retrieve verification record and populate order_ids[]. Not the billing key.

billing key component

Truv constructs Gate B composite key: (client_id):(loan_number):client_id from XAccess-Client-Id — NOT in payload. Gate B: structural dedup — loan already billed?

routing flag

true → CREATE billing event (Step 4 in §4.4) false → VOID billing event (Step 5 in §4.4) Drives Truv processing path.

billing anchor

funded_at = funding wire disbursement timestamp in BSS (UTC). Not closing date. Billing period anchor.

Gate A key

Human-readable format (recommended): (client_id)-(loan_number)-(date)-v1 Example: bss_wc_001-2026-WC-008812-2026-06-03-v1 Gate A: exact-retry dedup — seen before? → 409 Reversal: append -reversal-v1. TTL: 90 days.

loanback flag

first_mortgage | second_mortgage. Optional. Required for 80/10/10 piggyback identification (2x PCL).

if funded=false

rescission | funding_error | borrower_withdrawal | other. Required when funded=false.

Field	Type	Required	Description
order_id	string (UUID)	Yes	Truv order identifier. Used to look up the verification record and populate order_ids[] on

			the billing event. Not the billing primary key — loan_number is. Must match an existing order in Truv.
loan_number	string	Yes	★ Billing key component. Truv constructs the composite billing PK as {client_id}:{loan_number} — client_id is derived from the X-Access-Client-Id auth header, so BSS does not need to send it separately. This prevents cross-lender collision when multiple BSS lenders share the same loan numbering scheme. All order_ids for this loan stored as metadata.
funded	boolean	Yes	true = loan funded (create billing event). false = reversal/rescission (void billing event).
funded_at	ISO 8601 datetime	Yes	Timestamp of funding event in Blue Sage (UTC). Billing period anchor. Must be the funded status transition time, not closing date.
idempotency_key	string	Yes	Blue Sage-generated per-event string. Gate A: exact-retry guard — same event fired more than once (BSS framework retry, network hiccup). TTL 90 days. Recommended format: {client_id}-{loan_number}-{funded_date}-v1 (e.g. bss_wc_001-2026-WC-008812-2026-06-03-v1). Reversal: append -reversal-v1. Important: the client_id prefix in this format is supplied by BSS for auditability only — it is not a security control. Gate A is a string-match lookup; Truv does not validate the client_id embedded in this key. The security-critical client_id is in the X-Access-Client-Id header, which Truv uses exclusively for Gate B composite key construction. BSS must generate a new key per event — never reuse.
loan_type	string (enum)	No	first_mortgage second_mortgage. Required for piggyback identification. Omit for standard single-lien loans.
reversal_reason	string (enum)	If funded=false	rescission funding_error borrower_withdrawal other. Required when funded=false.

3.5.3 Sample payload — happy path

Event name: loan.funded

```
{ "order_id": "ord_abc123xyz",
  "loan_number": "2026-WC-008812",
  "funded": true,
  "funded_at": "2026-06-03T14:22:00Z",
  "idempotency_key": "bss_wc_001-2026-WC-008812-2026-06-03-v1",
```

"loan_type": "first_mortgage" } Note: Truv billing store receives order_ids:["ord_abc123xyz"] (most-recent first). If a refresh order is created later and a subsequent funded notification arrives, Truv prepends the refresh order_id: order_ids:["ord_refresh_xyz", "ord_abc123xyz"].

3.5.4 Sample payload — reversal

Event name: loan.reversed

```
{ "order_id": "ord_abc123xyz",  
  "loan_number": "2026-WC-008812",  
  "funded": false,  
  "funded_at": "2026-06-03T14:22:00Z",  
  "idempotency_key": "bss_wc_001-2026-WC-008812-2026-06-03-reversal-v1",  
  "reversal_reason": "rescission" }
```

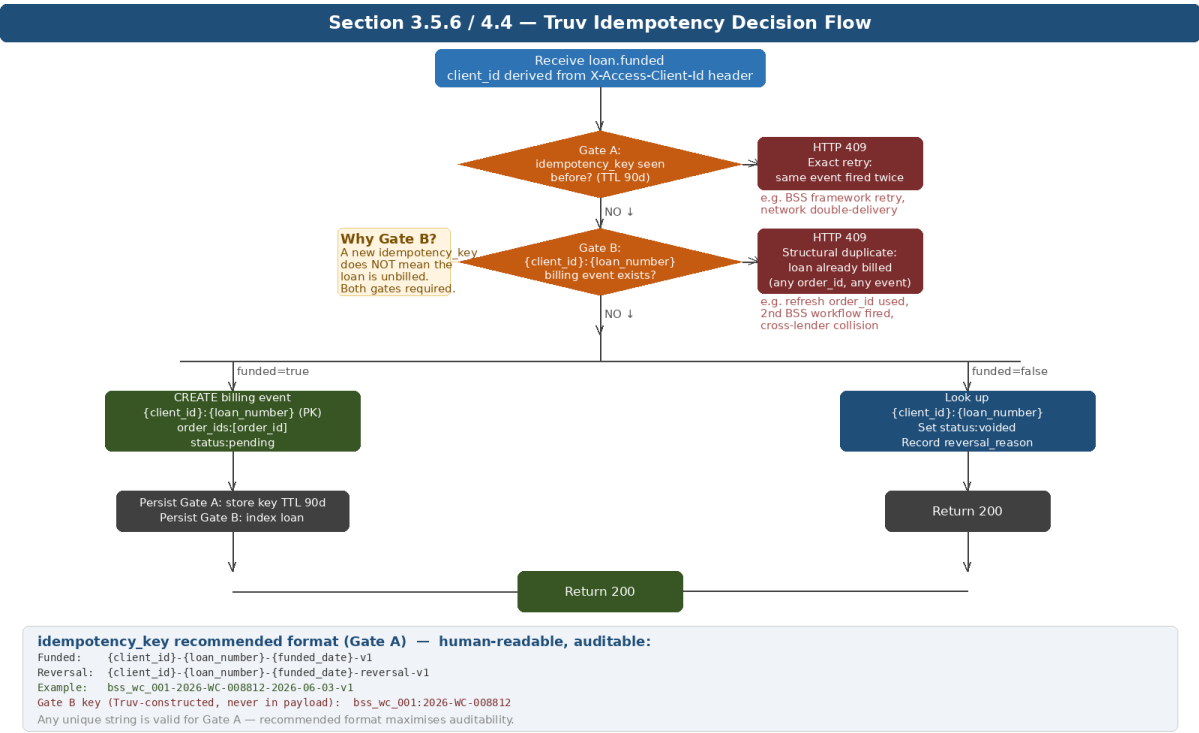
3.5.5 Truv response codes

HTTP status	Meaning and Truv action
200	Success — billing event created (funded=true) or voided (funded=false). Idempotency key stored.
401	Invalid auth credentials. Blue Sage must not retry without correcting credentials.
404	order_id not found in Truv. Alert Truv ops — indicates a sync issue between systems.
409	Duplicate. Gate A: idempotency_key seen before — same event was already processed (exact retry). Gate B: {client_id}:{loan_number} already has a billing event — loan was already billed, regardless of order_id or idempotency_key (cross-order or cross-workflow duplicate). No second billing event created in either case. BSS must not retry on 409.
400	Validation error — missing required field, invalid funded_at format, or unknown loan_type value. Do not retry without correcting payload.
5xx	Truv internal error. Retry with exponential backoff: 1 min, 5 min, 30 min. After 3 failures, alert Truv ops.

3.5.6 Idempotency design

Two-gate deduplication. Gate A (idempotency_key): catches the exact-retry scenario — same event fired more than once because BSS's event framework retried or a network hiccup caused a double delivery. The key is the same on every retry of the same event, so Truv returns 409 immediately. Gate B ({client_id}:{loan_number} composite): catches the structural duplicate scenario — same loan already billed, regardless of whether the incoming event has a new idempotency_key. This covers: (a) a refresh order_id used instead of the original, (b) two BSS workflows both firing for the same funded loan, (c) cross-lender collision where two lenders share the same loan_number. client_id for Gate B is derived from X-Access-Client-Id — BSS does not send it in the payload. This is intentional: the billing dedup key must be tied to a verified authenticated identity, not a payload value BSS could misconfigure. Note: the idempotency_key (Gate A) may include client_id as a human-readable prefix for auditability (e.g. bss_wc_001-2026-WC-008812-2026-06-03-v1), but Truv does not validate or use that embedded value — Gate A is a pure string-match lookup. Gate A alone is not sufficient: a new idempotency_key does not mean the loan has not already been billed. Both gates are required. Example: if a refresh order was created after the original VOIE order (the Truv

API creates a new order_id for each refresh). Using order_id alone as the billing key would allow double-billing if Blue Sage sends the funded notification with the refresh order_id rather than the original. Truv processing logic (full sequence in Section 4.4 Steps 1–7): Step 1 verifies X-BSS-Webhook-Sign before any billing logic runs.



Truv stores the idempotency_key with TTL 90 days (Gate A store). Truv maintains a billing index keyed on {client_id}:{loan_number} for permanent Gate B dedup. Blue Sage must generate a new key per funding event — never reuse. Recommended format: {client_id}-{loan_number}-{funded_date}-v1 for funded events; {client_id}-{loan_number}-{funded_date}-reversal-v1 for reversal events. BSS must send the same loan_number consistently across the funded notification and any subsequent reversal — loan_number is the stable Gate B anchor regardless of which order_id is used. The billing event stores order_ids[] as an array of all Truv order IDs associated with the loan, ordered most-recent-first. Blue Sage sends the most recent active order_id in the webhook payload; Truv stores it as the first element of order_ids[]. If subsequent funded notifications arrive for the same loan_number (e.g. a refresh order_id is used), Truv prepends the new order_id to the array. The array is for research, audit, and reconciliation purposes only — it has no effect on billing logic or deduplication. Billing deduplication (Gate B) is keyed on {client_id}:{loan_number} — client_id derived from the auth header.

3.5.7 Retry policy (Blue Sage outbound)

If Truv returns 5xx, Blue Sage's webhook framework retries with exponential backoff: 1 minute, 5 minutes, 30 minutes (3 attempts). Blue Sage must NOT retry on 409 (duplicate) or 4xx (client error). After 3 failed attempts, Blue Sage ops is alerted and Truv's reconciliation backstop (Path D) provides the safety net.

Cross-reference: The funded notification is designed as a webhook (Blue Sage → Truv) to align with the case study brief's requirement for 'webhook contracts, both directions' and with enterprise LOS architectural patterns. Full design rationale and REST POST fallback are in Section 4.1. The four funded scenarios with step-by-step sequencing are in Section 4.0. The reconciliation backstop for missed notifications is in Section 6.1.

4. Loan-Funded Notification Design

4.0 Funded notification — all scenarios

Four scenarios with continuous step numbering: happy path (steps 1–8), duplicate/idempotent (steps 9–11), reversal (steps 12–15), piggyback 2x PCL (steps 16–20).

Step	Actor	Message / Action	Notes
Scenario 1 — Happy path: loan funded			
1	Blue Sage	Loan status transitions to funded	Note executed; funded_at timestamp recorded in Blue Sage. Distinct status from closed.
2	Blue Sage → Truv	Webhook: loan.funded (order_id, loan_number, funded_at, idempotency_key, funded:true)	Primary: Blue Sage event framework → Truv subscription endpoint. Fallback: POST /v1/orders/{id}/funded/
3	Truv	Verify X-BSS-Webhook-Sign: HMAC-SHA256(raw_body, signing_secret). Constant-time compare. 401 if mismatch — stop.	Step 1 of §4.4. First check — no billing logic runs until signature passes.
4	Truv	Validate X-Access-Client-Id + X-Access-Secret	
5	Truv	Look up order_id	401 if invalid.
6	Truv	Check idempotency_key store, then check loan_number billing index	404 if not found — alert Truv ops. Note: order_id is used to retrieve the verification record only. If loan has multiple order_ids (original + refresh), Blue Sage should send the most recent active order_id.
			Gate A: idempotency_key seen before? → 409, stop. Catches: same event retried (network/framework retry). Gate B: {client_id}:{loan_number} already has a billing event? → 409, stop. Catches: same loan already billed via any event — refresh order_id, second workflow, or cross-lender collision. client_id derived from auth header.

7	Truv → Billing Store	Create billing event { {client_id}:{loan_number} (composite PK), order_ids:["ord_latest",...,"ord_original"] (most-recent first), funded_at, status:pending }	
8	Truv	Persist Gate A: store idempotency_key TTL 90d. Persist Gate B: index {client_id}:{loan_number} → billing_event_id (permanent). order_ids[] initialized as [triggering_order_id] (most-recent-first).	
9	Truv → Blue Sage	HTTP 200 – billing event created	
Scenario 2 — Duplicate notification (webhook retry / network retry)			
9	Blue Sage → Truv	Webhook: loan.funded (same idempotency_key)	Event framework retry or Blue Sage resends.
10	Truv	idempotency_key already in store	No billing event created.
11	Truv → Blue Sage	HTTP 409 – duplicate, billing event unchanged	
Scenario 3 — Funding reversal			
12	Blue Sage → Truv	Webhook: loan.reversed (funded:false, reversal_reason)	
13	Truv	Look up billing event for {client_id}:{loan_number} composite key	
14	Truv → Billing Store	Update billing event → status:voided	
15	Truv → Blue Sage	HTTP 200	
Scenario 4 — Piggyback: closed-end second mortgage (2x PCL)			

16	Blue Sage	First lien already funded and billed (prior webhook)	Separate billing event on record for first lien loan_number.
17	Blue Sage → Truv	Webhook: loan.funded (loan_number:2nd lien, new idempotency_key)	Different loan_number on same order_id = distinct billable event.
18	Truv	{client_id}:{loan_number_2} not seen before → new billable event	Each lien has a distinct loan_number. Truv constructs {client_id}:{loan_number} and {client_id}:{loan_number_2} as separate composite PKs → two distinct billing events. idempotency_key is per-event; the composite key is the stable cross-lender, cross-order dedup anchor.
19	Truv → Billing Store	Create 2nd billing event {client_id}:{loan_number_2} (composite PK), order_ids:["ord_latest"] (most-recent first), funded_at, status:pending }	2x PCL confirmed assumption. Fallback to 1x requires Truv commercial confirmation.
20	Truv → Blue Sage	HTTP 200	

4.1 Webhook vs. REST POST — design decision and rationale

Decision: Blue Sage → Truv funded notification uses a webhook

This PRD assumes Blue Sage has an outbound webhook/event framework — a reasonable assumption given it is a cloud-native, API-driven enterprise LOS that integrates with title, MI, flood, and other vendors via event-based notifications. If this assumption cannot be confirmed, the fallback is a REST POST (see Section 4.1.2). The design decision is documented explicitly because the homework brief specifies 'webhook contracts, both directions' — this is not a red herring.

4.1.1 Why webhooks for the funded event (primary design)

A webhook subscription is architecturally preferable to a REST POST callout for the funded notification for the following reasons:

- **Durability:** Enterprise LOS webhook frameworks (e.g. MuleSoft, Boomi, or a native event bus) provide guaranteed delivery semantics — events are persisted and retried automatically until acknowledged. A REST callout is only as reliable as the application code that fires it.
- **Operational resilience:** Blue Sage's webhook framework handles retry, backoff, and delivery receipts natively. Truv does not need a polling backstop if the event framework guarantees delivery. The polling backstop (Path D) is retained as a secondary safety net regardless.
- **Vendor-agnostic architecture:** Blue Sage already publishes loan state events to other vendor subscribers (title, MI, flood). Adding Truv as a subscriber to a 'loan.funded' event is a configuration change, not a development project. No new Blue Sage callout code required.

- Compliance and audit: Webhook frameworks produce an immutable event log with delivery receipts per event. This is preferable to application-level logs for a billing-critical event.
- Enterprise lender preference: In practice, large enterprise LOS shops (confirmed by ICE/Encompass client patterns) strongly prefer webhook subscription models over vendor-specific REST callout integrations. Webhooks decouple the LOS from vendor API contract changes.
- Bidirectional contract: The case study brief explicitly calls for 'webhook contracts, both directions.' This design satisfies that requirement literally and architecturally.

4.1.2 REST POST fallback (if Blue Sage webhook framework cannot be confirmed)

If Blue Sage does not support outbound webhook subscriptions, the funded notification reverts to a REST POST callout:

- Blue Sage makes a direct POST to a new Truv endpoint: POST /v1/orders/{id}/funded/
- Blue Sage owns the retry logic on 5xx responses.
- Truv deduplicates via Gate A (idempotency_key — exact retry) and Gate B ({client_id}:{loan_number} — structural dedup: same loan billed via any event, any order_id, any lender). Both gates apply identically in REST fallback mode.
- The polling backstop (Path D) becomes mandatory rather than advisory.

Research action (Note 1 in Section 10): Confirm with Blue Sage engineering whether their platform supports outbound webhook subscriptions for loan state change events. This single answer determines which design path is implemented. The funded notification schema (Section 4.2) is identical in both cases — only the delivery mechanism differs.

4.2 Request schema — loan-funded notification payload

{client_id}:{loan_number} is the composite billing key (v1.0 requirement) — client_id is derived by Truv from the X-Access-Client-Id header; BSS does not include it in the payload. order_id is metadata — Truv uses it to look up the verification record and populate order_ids[]. For the REST POST fallback, order_id also appears as a path parameter: POST /v1/orders/{order_id}/funded/

Field	Type	Required	Notes
order_id	string (UUID)	Yes	Most recent active Truv order_id on the loan. Metadata — Blue Sage should send the most recent order_id (original or refresh, whichever is current). Truv uses it to look up the verification record and populate order_ids[] on the billing event. Billing PK is loan_number.
loan_number	string	Yes	Bluesage internal loan number; stored by Truv for reconciliation
funded	boolean	Yes	true = funded; false = reversal/rescission
funded_at	ISO 8601 datetime	Yes	Timestamp of funding event in Bluesage (UTC). Used for billing period.
idempotency_key	string	Yes	Gate A: exact-retry guard. BSS generates one key per event; same key on all retries. Truv returns 409 if seen before (TTL 90 days). Catches: same event fired twice. Does NOT catch: same loan billed via a

Field	Type	Required	Notes
			different event — that is Gate B. Recommended format: {client_id}-{loan_number}-{funded_date}-v1 (e.g. bss_wc_001-2026-WC-008812-2026-06-03-v1). Reversal: bss_wc_001-2026-WC-008812-2026-06-03-reversal-v1. Note: client_id in this key is BSS-supplied for auditability — not a security control. Truv does not validate it here. The authoritative client_id is always the X-Access-Client-Id header (used for Gate B).
reversal_reason	string (enum)	If funded=false	Values: rescission funding_error borrower_withdrawal other

4.3 Sample request

```
POST /v1/orders/ord_abc123xyz/funded/
X-Access-Client-Id: {client_id} | X-Access-Secret: {access_key}
{ "order_id": "ord_abc123xyz", "loan_number": "2026-WC-008812",
  "funded": true, "funded_at": "2026-06-03T14:22:00Z",
  "idempotency_key": "bss_wc_001-2026-WC-008812-2026-06-03-v1" }
```

4.4 Truv processing logic on receipt

- Step 1 — Verify X-BSS-Webhook-Sign header: compute HMAC-SHA256(raw_request_body, bss_signing_secret) and compare to header value. Reject with 401 if mismatch — stop, do not process. This is the first check before any other validation.
- Step 2 — Validate credential headers (X-Access-Client-Id + X-Access-Secret). Reject with 401 if invalid.
- Step 3 — Look up order_id. Return 404 if not found. order_id is used to retrieve the verification record only — not as the billing deduplication key.
- Step 4 — Deduplication (two gates). Gate A: idempotency_key seen before? Yes → 409, stop. Gate A catches: exact retry — same event fired twice (network hiccup, BSS event framework retry). Gate B: {client_id}:{loan_number} billing event exists? Yes → 409, stop. Gate B catches: structural duplicate — same loan already billed regardless of order_id or idempotency_key (refresh order, second BSS workflow, cross-lender collision). client_id derived from auth header.
- Step 5 — funded=true: create billing event { composite_key:{client_id}:{loan_number} (PK), order_ids:[triggering_order_id] (most-recent-first), funded_at, status:pending }. Return 200.
- Step 6 — funded=false: look up billing event by {client_id}:{loan_number} composite key. If found → set status:voided, record reversal_reason, return 200. If not found (reversal before funded event) → queue for 72h; apply immediately when funded event arrives.
- Step 7 — Persist: store idempotency_key (TTL 90 days); update billing index {client_id}:{loan_number} → billing_event_id; if billing event already exists, prepend incoming order_id to order_ids[] (keeps array most-recent-first).

4.5 Truv endpoint specification — POST /v1/orders/{order_id}/funded/

This section defines what the Truv engineering team builds. The funded notification endpoint is net-new — it does not exist today. Truv eng owns the receiver; BSS eng owns the sender. Everything below maps directly to the 7-step processing logic in Section 4.4.

4.5.1 Endpoint summary

Field	Value	Notes
Method	POST	Standard HTTP POST
URL	/v1/orders/{order_id}/funded/	order_id is path param — used for order lookup only (metadata), not billing key
Base URL	https://prod.truv.com/v1/	Same base as all Truv API endpoints
Sandbox	https://sandbox.truv.com/v1/	Use for TC-01 through TC-09
Owner	Truv eng	Net-new endpoint — does not exist today

4.5.2 Required request headers

Header	Value / Format	Notes
X-BSS-Webhook-Sign	hex(HMAC-SHA256(raw_body, signing_secret))	Step 1. Payload integrity. Truv verifies before any other processing. 401 if mismatch or missing.
X-Access-Client-Id	BSS lender client identifier	Step 2. Used to construct Gate B composite key {client_id}:{loan_number}. 401 if invalid.
X-Access-Secret	BSS lender access secret	Step 2. Credential auth. 401 if invalid.
Content-Type	application/json	Required on all POST requests.

Signature format: lowercase hex-encoded SHA256 digest. Example: X-BSS-Webhook-Sign: 3a7bd3e2360a3d29eea436cfb7e44c735d117c42d1c1835420b6b9942dd4f1b. Truv must use constant-time comparison (e.g. hmac.compare_digest in Python) to prevent timing attacks.

4.5.3 Endpoint handler — pseudo-code stub

This stub is the buildable translation of Section 4.4 Steps 1–7. Truv eng implements this function; all inputs, outputs, and error codes are fully specified in this PRD.

```
POST /v1/orders/{order_id}/funded/

def handle_funded_notification(order_id, headers, raw_body, payload):

    # Step 1 — Verify payload signature (X-BSS-Webhook-Sign)
    expected_sig = hmac_sha256_hex(raw_body, signing_secret_store[headers.client_id])
    if not constant_time_equal(headers.bss_webhook_sign, expected_sig):
        return 401 # Invalid signature — stop

    # Step 2 — Validate credentials (X-Access-Client-Id + X-Access-Secret)
    if not credential_store.valid(headers.client_id, headers.access_secret):
        return 401 # Invalid credentials — stop

    # Step 3 — Look up order_id (metadata only — not billing key)
    order = order_store.get(order_id)
    if not order: return 404 # order_id not found
```

```

# Step 4 – Deduplication: Gate A then Gate B
if idempotency_store.seen(payload.idempotency_key):
    return 409 # Gate A: exact retry
composite_key = f"{headers.client_id}:{payload.loan_number}"
if billing_index.exists(composite_key):
    return 409 # Gate B: loan already billed

# Steps 5 / 6 – Create or void billing event
if payload.funded:
    billing_store.create(
        pk            = composite_key,
        order_ids     = [order_id], # most-recent-first array
        funded_at     = payload.funded_at,
        status        = "pending"
    )
else:
    event = billing_store.get(composite_key)
    if event:
        billing_store.void(composite_key, payload.reversal_reason)
    else:
        reversal_queue.hold(composite_key, payload, ttl=72h)

# Step 7 – Persist
idempotency_store.set(payload.idempotency_key, ttl=90d)
billing_index.upsert(composite_key, prepend_order_id=order_id)

return 200

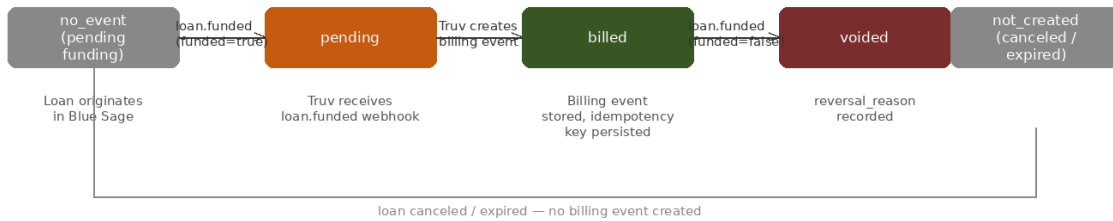
```

4.5.4 Response codes — Truv returns to BSS

HTTP Status	Meaning	BSS action
200	Success	Billing event created (funded=true) or voided (funded=false). Idempotency key stored.
401	Auth failure	Invalid X-BSS-Webhook-Sign (Step 1) or invalid credentials (Step 2). BSS must not retry without fixing.
404	order_id not found	order_id in path does not match any Truv order. Alert Truv ops – indicates BSS/Truv sync issue.
409	Duplicate	Gate A (idempotency_key seen) or Gate B ({client_id}:{loan_number} already billed). Do not retry.
400	Validation error	Missing required field, invalid funded_at format, unknown loan_type. Fix payload before retry.
5xx	Truv internal error	BSS retries with backoff: 1m, 5m, 30m. After 3 failures, alert Truv ops. Path D backstop activates.

5. Loan-State Taxonomy and Billing Mapping

Section 5 — PCL Billing Event State Machine



Bluesage loan state	PCL billing action	Notes
originated	No action	Order created in Truv; no billing event
in_underwriting	No action	VOIE completed; verification cost accrues but PCL not triggered
clear_to_close	No action	May trigger refresh order; still no billing
funded	CREATE billing event	Single billing event regardless of # of verifications on order
funded → reversal	VOID billing event	POST /funded/ with funded=false. Billing event status → voided.
closed	No billing event	Closing and funding are distinct workflow stages in Blue Sage. In most mortgage transactions, closing (note execution at the settlement table) precedes or coincides with funding (wire disbursement). PCL billing fires on the funded transition only — closing status does not trigger a new billing event. Exact sequencing of closing vs. funded status transitions in Blue Sage must be confirmed with Blue Sage eng — see Section 10 Note 2.
canceled / expired	No billing event	Loan never funded — zero PCL charge

PCL billing fires once per {client_id}:{loan_number} composite key regardless of verification product count or refresh cycles. Piggyback (80/10/10): each lien has a distinct loan_number; Truv constructs separate composite keys and generates separate billing events (2x PCL). HELOCs not supported by Truv. Loan_number is the billing anchor in all cases.

6. Error Handling and Reconciliation

Scenario	Truv behavior	Reconciliation mechanism
Duplicate funded notification	Return 200; skip billing	Gate A (idempotency_key): exact-retry dedup — same event fired twice. Gate B ({client_id}:{loan_number}): structural dedup

Scenario	Truv behavior	Reconciliation mechanism
		— same loan already billed via any event. Log duplicate with gate that fired for audit.
Reversal before funded event	Queue reversal; wait for funded	Hold reversal record 72h. If funded arrives, apply reversal immediately. If not, discard.
Out-of-order events	Process by funded_at timestamp	Truv orders events by funded_at, not receipt time.
Missing funded notification	Truv polling backstop (T+24h)	Truv polls GET /bluesage/loans/{id}/status daily for orders in completed state >24h with no funded event. Bluesage must expose this endpoint.
Webhook delivery failure	Retry with exponential backoff	3 retries: 1m, 5m, 30m. After 3 failures, alert Truv ops. Endpoint must respond <1s.
Funded with no matching order	Return 404; log alert	Truv support notification. Bluesage must investigate order_id mismatch.

6.1 Reconciliation backstop — Blue Sage polling endpoint

Truv will poll this endpoint daily for orders in completed state with no funded event received after 24 hours:

```
GET /bluesage/loans/{loan_number}/status → { loan_number, status, funded_at, order_id }
```

Blue Sage engineering must confirm whether this endpoint exists today or needs to be built. If net-new, estimate 1–2 sprint build time; Truv PM to track as a dependency.

7. Sandbox Test Plan

TC#	Scenario	Credentials	Expected outcome
TC-01	Happy path — single employer, funded	goodlogin / goodpassword	Order completes → funded notification → billing event created
TC-02	Multiple employers, funded	multiple.employments / goodpassword	Single billing event despite 2 verifications on same order_id
TC-03	Loan never funds (canceled)	goodlogin / goodpassword	Order completed; no funded notification sent; zero PCL charge
TC-04	Duplicate funded notification	goodlogin / goodpassword	Second POST returns 200; billing event count remains 1
TC-05	Funding reversal	goodlogin / goodpassword	funded=false POST voids billing event; status → voided
TC-06	MFA / re-verification required	any / mfa	task-status-updated fires error; refresh order triggered; re-verify completes

TC#	Scenario	Credentials	Expected outcome
TC-07	No-data provider	any / goodpassword (no-data employer)	Order status = no_data; fallback to doc upload; no automated billing impact
TC-08	Webhook timeout / retry	goodlogin / goodpassword	Endpoint returns >1s; Truv retries 3x with backoff; final delivery confirmed
TC-09	Reversal arrives before funded	goodlogin / goodpassword	Truv queues reversal; funded arrives within 72h; reversal applied immediately

All test cases must pass in sandbox before production credentials are issued. TC-04 (duplicate idempotency) and TC-05 (reversal) must each be run 3 independent times to confirm deterministic behavior.

8. Go-Live Checklist

Validation step	Owner	Go/no-go criterion
Sandbox TC-01 through TC-09 all pass	Truv + both eng teams	Zero failures
HMAC-SHA256 signature verified on all inbound webhooks — both Truv→LION (X-Webhook-Sign) and BSS→Truv (X-BSS-Webhook-Sign)	LION eng + Blue Sage eng	No rejected sigs in 48h soak on both directions
Funded notification idempotency test (TC-04) passes 3 independent runs	Blue Sage eng	Billing event count = 1 in all 3 runs
Reversal flow (TC-05) validated end-to-end in staging	Blue Sage eng + Truv billing	Billing event status = voided confirmed
Polling backstop endpoint live and tested (GET /bluesage/loans/{id}/status)	Blue Sage eng	Truv polling confirms funded status correctly
Webhook endpoint response time <1s under load (50 concurrent)	LION eng + Blue Sage eng	p99 <800ms
Production credentials rotated and stored in secrets manager: client_id, access_secret, AND X-BSS-Webhook-Sign signing secret (not env vars)	Both eng teams	Security sign-off
Truv CS confirms lender configuration (products, data_sources, webhook endpoint) and has provisioned BSS with client_id, access_secret, AND webhook signing secret	Truv CS	Lender config doc signed off including signing secret issuance confirmed
Rollback plan documented: revert to manual Blue Sage one-click flow	Truv PM	Rollback runbook approved

8.1 Rollback plan

If a critical defect is discovered post-launch (funded notifications failing, billing events not created, or double-billing detected):

1. Blue Sage disables the funded notification callout in their loan closing workflow.
2. Lender reverts to manual Truv one-click order flow in Blue Sage (existing capability — no build required).
3. Truv ops manually reconciles billing events for the affected window using `client_id` + `loan_number` + `funded_at` from Blue Sage export.
4. Fix is developed, retested against TC-01 through TC-09, and re-deployed. Re-enablement requires Truv PM sign-off.

9. Open Questions and Punted Decisions

The following will be resolved with engineering tech leads in the next sprint. They are explicitly not blocking PRD approval — engineering can begin building against this spec while these are resolved in parallel.

Open question	Owner	Impact if unresolved
What is the SLA for Truv to process a funded event and create a billing record?	Truv billing	Needed for lender reporting and month-end reconciliation
Will LION require a co-borrower verification path (2 borrowers, 1 order)?	LION PM	Embedded orders support multiple employers but single borrower; co-borrower needs separate <code>order_id</code>
Does Blue Sage expose a loan status polling endpoint today, or does it need to be built?	Blue Sage eng	Required for reconciliation backstop; 1–2 sprint build if net-new
CONFIRMED v1.0 REQUIREMENT — Inbound BSS→Truv payload signing (X-BSS-Webhook-Sign, HMAC-SHA256) is required for production go-live. Truv issues BSS a webhook signing secret at onboarding; BSS signs each <code>loan.funded</code> payload; Truv verifies as Step 1 in Section 4.4 before any billing logic. Delivers payload integrity + symmetry with Truv→LION model + enterprise webhook reliability. Action: Truv eng to implement verification and add signing secret to CS onboarding runbook.	Truv eng + Blue Sage eng	v1.0 requirement. Implement before first production lender onboarding. Truv CS runbook must provision signing secret alongside <code>client_id</code> and <code>access_secret</code> .

Appendix I — AI in This Work: Reflection

What AI compressed

- API reconnaissance: fetching and synthesizing docs.truv.com (orders, LOS custom implementation, POS custom implementation, testing credentials, Blue Sage guide) in under 30 minutes rather than 2–3 hours of manual reading.
- PRD scaffolding and table generation: the structure, section ordering, and table schemas were generated rapidly. This freed time for the architectural and commercial design decisions.
- Schema design iteration: quickly modeling the funded notification payload, idempotency logic, and reversal handling across multiple passes.

Where judgment calls were mine

- Push-over-poll as the primary funded notification mechanism, with polling as a reconciliation backstop — not obvious from the docs, a design choice.
- Billing key: composite {client_id}:{loan_number} — retained in v1.0 as a billing correctness requirement, not a future-state feature. Cross-lender loan number collision is a production certainty in multi-lender BSS deployments. client_id is derived from the auth header; BSS payload is unchanged.
- Single PCL billing event per {client_id}:{loan_number} composite key regardless of refresh cycle count — one billing event per funded loan per lender client. Composite key is v1.0.
- The funded_at timestamp as the billing period anchor (not disbursement date or closing date) — a mortgage domain call based on how funding events are recorded in LOS platforms.
- Framing the reconciliation backstop as a Blue Sage polling endpoint (not a Truv-initiated scan) — puts the source of truth with the system that owns the loan state.
- The 72-hour queue window for reversals-before-funded — a judgment call balancing lender operational reality against Truv's billing cycle.

Tradeoffs made

- Revised from REST POST to webhook subscription for the funded notification after re-examining the case study brief. The brief explicitly calls for "webhook contracts, both directions" — this was initially interpreted as potentially a red herring, but on reflection it is the correct architectural pattern for an enterprise LOS. Blue Sage, as a cloud-native API-driven platform, almost certainly has an outbound event framework. The webhook design is more durable, audit-friendly, and consistent with how enterprise lenders (confirmed via ICE/Encompass client experience) prefer to integrate with verification vendors. REST POST fallback is documented in Section 4.1.2 for cases where the webhook assumption cannot be confirmed.

Correspondent and wholesale channel context: In the wholesale channel, the mortgage broker originates the loan and submits it to the lender (via LION and Blue Sage), who underwrites and funds. The lender holds the note at funding, then typically sells it to an investor. In the correspondent channel, the originating lender funds the loan with its own capital (table-funded), then sells the closed loan to an investor or aggregator. In both channels, the PCL billing trigger is the same: the funded transition in Blue Sage when the lender disburses the wire. The key distinction for this integration is that correspondent lenders may have a gap of 1-3 business days between closing (note signing) and funding (wire disbursement) in non-table-funded transactions — the funded notification must fire on the funded status transition, not the closing transition. Exact Blue Sage status field names for both transitions must be confirmed with Blue Sage engineering.

Appendix II — API Call Index

All API calls, webhooks, and internal actions keyed to Section 1.2 step numbers. Use the Spec ref column to navigate to the full contract for any step.

Step	Actor	Method	Endpoint / Event	Dir	Spec ref
PATH A — Borrower Verification (Steps 1-8)					
1	LION → Truv	POST	/v1/orders/	→	\$2.1, \$4.3
2	Truv → LION	HTTP 201	{ order_id, bridge_token }	←	\$2.2
3	LION → Borrower	JS init	TruvBridge.init(bridge_token)	→	\$2.3
4	Borrower → Truv	(Bridge)	payroll auth — black box	→	—
5	Truv → LION	WEBHOOK	task-status-updated	←	\$2.2, \$3.1
6	Truv → LION	WEBHOOK	order-status-updated	←	\$2.2, \$3.1
7	LION → Truv	GET	/v1/orders/{order_id}/	→	\$2.1, \$2.2
8	Truv → Blue Sage	(existing)	direct report delivery	→	\$1.3 Path A
PATH B — Loan-State Events & PCL Billing ★ Net-new (Steps 9-16)					
9	Blue Sage (internal)	—	loan status → funded	—	\$5
10	Blue Sage (internal)	—	funded_at timestamp recorded	—	\$5
11	BSS → Truv	WEBHOOK	loan.funded	→	\$3.5, \$4.2
12	Truv (internal)	—	validate + dedup (Steps 1-3)	—	\$4.4 Steps 1-3
13	Truv → Billing	—	CREATE billing event	—	\$4.4 Step 4
14	Truv → BSS	HTTP 200	billing event created	←	\$3.5.5
15	BSS → Truv	WEBHOOK	loan.reversed (if applicable)	→	\$3.5.4, \$4.0
16	Truv (internal)	—	VOID billing event	—	\$4.4 Step 5
PATH C — Refresh Verification (Steps 17-20b)					
17	Blue Sage (internal)	—	LO triggers refresh	—	\$5
18	BSS → Truv	POST	/v1/orders/{id}/refresh/	→	\$2.1
19	Truv → Borrower	(Bridge)	new Bridge session	→	\$1.3 Path C
20a	Truv → LION	WEBHOOK	order-status-updated	←	\$2.2, \$3.1
20b	Truv → Blue Sage	(existing)	refreshed report delivery	→	\$1.3 Path C
PATH D — Reconciliation Backstop (Steps 21-23)					
21	Truv (internal)	—	T+24h poll trigger	—	\$6
22	Truv → BSS	GET	/bluesage/loans/{id}/status	→	\$6.1
23	BSS → Truv	HTTP 200	{ status, funded_at }	←	\$6.1
SUPPLEMENTARY — Duplicate / Reversal / Piggyback / Registration (S1-S4, R1-R3)					
S1	BSS → Truv	WEBHOOK	loan.funded (duplicate)	→	\$4.0 Scenario 2
S2	Truv → BSS	HTTP 409	duplicate — no billing	←	\$3.5.5
S3	BSS → Truv	WEBHOOK	loan.funded (2nd lien piggyback)	→	\$4.0 Scenario 4
S4	BSS → Truv	WEBHOOK	loan.reversed (reversal)	→	\$4.0 Scenario 3
R1	LION → Truv	POST	/v1/webhooks/	→	\$2.3
R2	Truv → LION	HTTP 201	{ webhook_id, secret }	←	\$2.3

R3	BSS → Truv	POST	/v1/orders/{id}/funded/ (fallback)	→	\$4.1.2
-----------	------------	------	---------------------------------------	---	---------

→ = outbound from left actor ← = response/inbound - = internal action \$ = Section
Steps 1-23 correspond to Section 1.2 sequence table. S-rows = scenario variants. R-rows = one-time setup.
Full request/response shapes: \$2.1 (endpoints), \$2.2 (fields), \$3.5.2 / \$4.2 (payload schema), \$4.3 (sample request).

